

MySQL: guida rapida alle query

1. SQL in breve

SQL (*Structured Query Language*) è il linguaggio usato per comunicare con un database relazionale.

- **DDL** definisce la struttura: **CREATE**, **ALTER**, **DROP**.
- **DML** modifica i dati: **INSERT**, **UPDATE**, **DELETE**.
- **DQL** interroga i dati: **SELECT**.

In questo esercizio si usa soprattutto **SELECT**: si descrive **quale risultato** si vuole ottenere, mentre MySQL decide come recuperarlo.

2. Preparare il database in MySQL Workbench

1. Aprire la connessione al server MySQL.
2. Nel pannello **Schemas**, creare uno schema chiamato **db_university**.
3. Fare doppio clic sullo schema: il nome in grassetto indica che è quello attivo.
4. Aprire il file **.sql** fornito con l'esercizio usando **File > Open SQL Script**. Il file potrebbe trovarsi nella cartella principale del progetto oppure in una sottocartella.
5. Eseguire lo script con il pulsante a forma di fulmine.
6. Aggiornare il pannello **Schemas** e controllare che siano comparse le tabelle.

Comandi utili per verificare l'importazione:

```
SHOW DATABASES;  
USE `db_university`;  
SHOW TABLES;  
DESCRIBE `students`;
```

Se compare l'errore *No database selected*, eseguire **USE db_university**; oppure fare doppio clic sullo schema.

3. Conoscere il dataset

Prima di scrivere una query, individuare la tabella e le colonne che contengono i dati richiesti.

Tabella	Contenuto	Colonne utili
students	studenti	date_of_birth, enrolment_date, degree_id
courses	corsi	period, year, cfu, degree_id
exams	appelli	date, hour, course_id
degrees	corsi di laurea	level, department_id
departments	dipartimenti	id, name

Tabella	Contenuto	Colonne utili
teachers	insegnanti	phone, office_address
exam_student	voti degli studenti agli appelli	exam_id, student_id, vote

Per osservare pochi record senza riempire la schermata:

```
SELECT *  
FROM `students`  
LIMIT 10;
```

4. Struttura di una query

```
SELECT `colonna`, funzione(`altra_colonna`) AS `alias`  
FROM `tabella`  
WHERE condizione  
GROUP BY `colonna`  
HAVING condizione_sul_gruppo  
ORDER BY `colonna` ASC;
```

Non tutte le clausole sono sempre necessarie. L'ordine, quando sono presenti, va rispettato.

Buone abitudini:

- scrivere le parole chiave SQL in maiuscolo;
- andare a capo a ogni clausola;
- terminare ogni query con `;;`;
- racchiudere nomi di tabelle e colonne tra backtick: ``students``;
- dare alle colonne calcolate un nome leggibile con `AS`.

5. Selezionare e filtrare

Tutte le colonne:

```
SELECT *  
FROM `students`;
```

Solo alcune colonne:

```
SELECT `name`, `surname`, `date_of_birth`  
FROM `students`;
```

Filtrare le righe con `WHERE`:

```
SELECT *  
FROM `courses`  
WHERE `cfu` >= 6;
```

Operatori principali:

Operatore	Significato
=	uguale
<>	diverso
<, <=, >, >=	confronti
BETWEEN ... AND ...	compreso in un intervallo, estremi inclusi
IN (...)	uguale a uno dei valori elencati
LIKE	corrisponde a un pattern
IS NULL, IS NOT NULL	valore assente o presente

Unire più condizioni:

```
SELECT *  
FROM `courses`  
WHERE `year` = 2  
    AND `period` = 'II semestre';
```

Con **AND** devono essere vere entrambe le condizioni; con **OR** ne basta una. Usare le parentesi quando si combinano:

```
WHERE (`year` = 1 OR `year` = 2)  
    AND `cfu` >= 6
```

Valori mancanti

NULL significa "valore assente". Non si confronta con **=**:

```
WHERE `phone` IS NULL
```

Testo e pattern

```
WHERE `name` LIKE 'E%'
```

- % rappresenta zero o più caratteri;
- _ rappresenta un solo carattere.

6. Date, orari ed età

MySQL usa normalmente questi formati:

- data: '2020-06-20' (AAAA-MM-GG);
- ora: '14:00:00' (HH:MM:SS).

Si possono confrontare direttamente:

```
WHERE `date` = '2020-06-21'
AND `hour` >= '09:00:00'
```

Estrarre l'anno da una data:

```
YEAR(`enrolment_date`)
```

Quando una richiesta riguarda l'età, osservare l'anno di nascita con `YEAR()` e confrontarlo con l'anno corrente. Prima di scrivere la condizione completa, può essere utile visualizzare il dato calcolato:

```
SELECT `date_of_birth`, YEAR(`date_of_birth`) AS `birth_year`
FROM `students`;
```

7. Contare e calcolare

Le funzioni di aggregazione trasformano più righe in un risultato calcolato:

Funzione	Risultato
<code>COUNT(*)</code>	numero di righe
<code>COUNT(colonna)</code>	numero di valori non <code>NULL</code> nella colonna
<code>AVG(colonna)</code>	media
<code>SUM(colonna)</code>	somma
<code>MIN(colonna)</code>	valore minimo
<code>MAX(colonna)</code>	valore massimo

Esempio:

```
SELECT COUNT(*) AS `total_students`
```

```
FROM `students`;
```

Calcolare una media:

```
SELECT AVG(`vote`) AS `average_vote`  
FROM `exam_student`;
```

8. Raggruppare con GROUP BY

GROUP BY riunisce le righe che hanno lo stesso valore. La funzione di aggregazione viene poi calcolata separatamente per ogni gruppo.

Schema mentale: **“per ogni X, calcola Y”**.

```
SELECT `degree_id`, COUNT(*) AS `student_count`  
FROM `students`  
GROUP BY `degree_id`;
```

Questa query restituisce una riga per ogni **degree_id**, con il numero di studenti del gruppo.

Regola fondamentale: nella **SELECT**, ogni colonna che non è dentro una funzione di aggregazione deve comparire nel **GROUP BY**.

Per raggruppare per anno si può usare la stessa espressione sia nella **SELECT** sia nel **GROUP BY**:

```
SELECT YEAR(`date_of_birth`) AS `birth_year`, COUNT(*) AS `student_count`  
FROM `students`  
GROUP BY YEAR(`date_of_birth`)  
ORDER BY `birth_year` ASC;
```

WHERE filtra le singole righe **prima** del raggruppamento; **HAVING** filtra i gruppi **dopo** il calcolo:

```
SELECT `degree_id`, COUNT(*) AS `student_count`  
FROM `students`  
GROUP BY `degree_id`  
HAVING COUNT(*) > 100;
```

9. Ordinare non significa raggruppare

ORDER BY cambia soltanto l'ordine delle righe restituite:

```
SELECT `name`, `cfu`  
FROM `courses`  
ORDER BY `cfu` DESC;
```

- **ASC**: crescente, valore predefinito;
- **DESC**: decrescente.

GROUP BY, invece, crea gruppi sui quali calcolare **COUNT**, **AVG** e le altre aggregazioni. Le ultime quattro richieste dell'esercizio sono quindi query di raggruppamento, anche se nel testo sono raccolte sotto il titolo "Order By queries".

10. Metodo pratico per risolvere gli esercizi

Per ogni richiesta:

1. individuare la tabella corretta;
2. controllarne le colonne con **DESCRIBE** e qualche riga con **LIMIT**;
3. tradurre le parole della richiesta in SQL:
 - "tutti" → **SELECT ***;
 - "dove" / "che hanno" → **WHERE**;
 - "e" → **AND**;
 - "quanti in totale" → **COUNT**;
 - "per ogni" → **GROUP BY**;
 - "media" → **AVG**;
4. eseguire la query e confrontare il numero di risultati con quello indicato;
5. se il risultato non coincide, mostrare temporaneamente le colonne coinvolte e verificare valori, formato e operatori;
6. copiare nel file di consegna solo la query testata e funzionante.

Checklist finale:

- la query usa la tabella corretta;
- date e testi sono tra apici singoli;
- **NULL** è controllato con **IS NULL**;
- **AND** e **OR** rappresentano davvero la richiesta;
- una query "per ogni" contiene **GROUP BY**;
- ogni colonna non aggregata della **SELECT** è nel **GROUP BY**;
- il risultato e il numero di righe sono plausibili;
- ogni query termina con **;**.

Non servono **JOIN** per completare questa esercitazione: ogni informazione richiesta può essere ricavata direttamente da una singola tabella.